MicroBlaze Statistics Accelerator SoC

Lucas Aguirre, Alexander Aziz, Nahjay Battieste, Michael Girgis

Final implementation: MicroBlaze + custom AXI-Lite statistics accelerator + Vitis standalone application

This report describes the final SoC implementation of a MicroBlaze-based statistics accelerator for the AMD Urbana board. The system uses a custom RTL peripheral, a dual-port BRAM shared between software and hardware, and a Vitis standalone application that initializes data, starts the accelerator, polls for completion, and verifies the hardware results against a software reference model.

1. Introduction

The objective of the project is to integrate a custom hardware accelerator into a MicroBlaze SoC and to exercise that accelerator from board-resident software. The final design is not a Linux-based system. Instead, a standalone Vitis application runs directly on the MicroBlaze processor and communicates with the accelerator through a memory-mapped AXI-Lite register interface.

The accelerator operates on 32-bit unsigned integers stored in BRAM. For a programmable dataset length, it computes five outputs: a 64-bit sum of all values, the minimum value, the maximum value, the count of values strictly greater than a programmable threshold, and an internal cycle count that measures the hardware run while the accelerator is busy. The software uses the same dataset to compute a reference result and then compares the two sets of outputs.

2. System Overview

At the system level, the design centers on a MicroBlaze soft processor connected to an AXI SmartConnect interconnect. The interconnect provides access to the custom statistics accelerator, the AXI BRAM Controller, the UARTLite console peripheral, standard reset and interrupt infrastructure, and the external MIG DDR3 interface present in the board design. In the final project flow, the processor writes the working dataset into BRAM, programs the accelerator registers, starts execution, and reads the result registers after completion.

The custom accelerator is implemented as an AXI-Lite slave peripheral with an additional direct BRAM read interface. AXI-Lite is used only for control and status, while the dataset is fetched through the second port of the block memory generator. The MicroBlaze therefore controls the run through memory-mapped registers but does not need to stream the dataset through the control path. UARTLite is used by the Vitis application to print the benchmark and PASS/FAIL messages to the serial console.

Figure 1 shows the Vivado block design for the implemented SoC. The processor, SmartConnect fabric, AXI BRAM Controller, custom accelerator, and shared BRAM are all visible in the design. The BRAM is dual ported:

port A is connected to the AXI BRAM Controller for software access, while port B is connected directly to the accelerator for hardware reads.

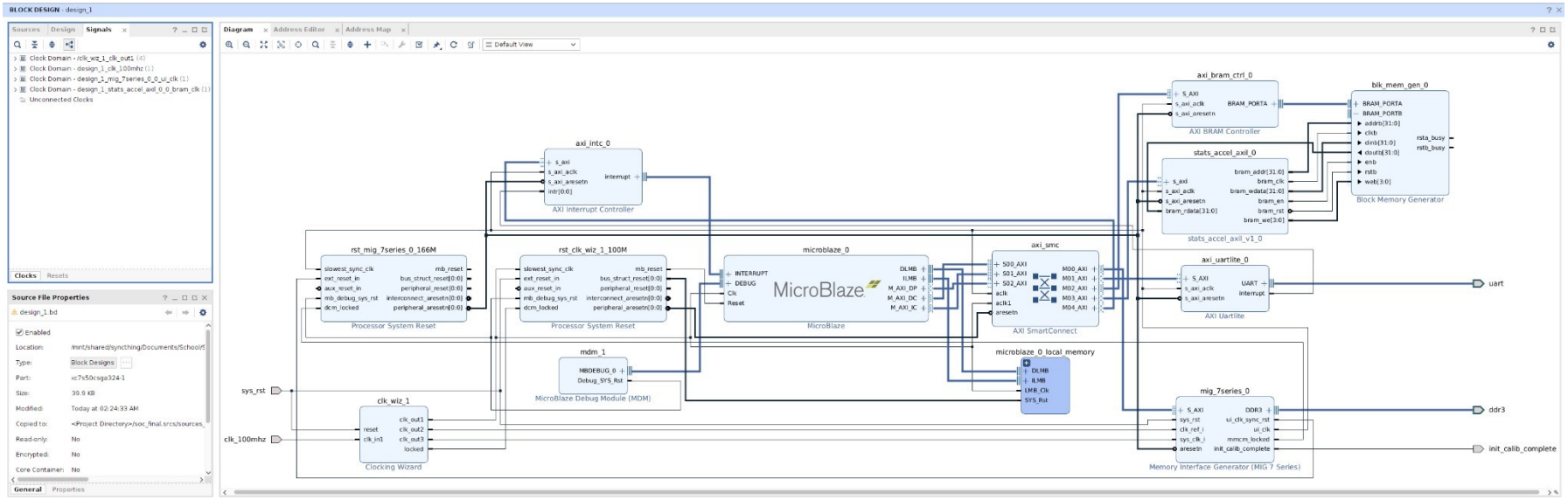

Figure 1. Vivado block design of the MicroBlaze statistics accelerator system. MicroBlaze accesses the control registers through AXI, writes the dataset through the AXI BRAM Controller, and the accelerator reads the same BRAM through the second memory port.

3. Hardware Architecture

The hardware organization follows a shared-memory control model. Software first populates the BRAM with the dataset. In the provided Vitis program, the BRAM base address is 0xC0000000, and each element is written as a 32-bit word at BRAM_BASE + 4*i. Those writes go through the MicroBlaze data-side AXI path, through SmartConnect, through the AXI BRAM Controller, and finally into port A of the block memory generator.

Once the dataset is resident in BRAM, the processor programs the accelerator through its AXI-Lite register file. The accelerator base address used by the application is 0x00008000. Software writes LENGTHWORDS and THRESHOLD, clears a previous done indication if needed, and then writes the start control bit. From that point on, the accelerator reads the dataset directly from BRAM port B without CPU intervention.

This data path is visible in the RTL wrapper. The AXI-Lite block drives only control registers and status reads. The BRAM-facing logic instead exposes a separate address, enable, clock, and data interface. The wrapper fixes the BRAM write enable to all zeros and the BRAM write data to zero, so the accelerator is strictly readonly with respect to dataset memory. Results are returned to software by reading the accelerator status registers after the run completes.

The address assignments shown later in Figure 2 confirm this split. The BRAM occupies an 8 KB region beginning at 0xC0000000, while the accelerator register block is mapped at 0x00008000 with a 4 KB AXILite aperture. The application uses only a small portion of the accelerator aperture because the implemented register file ends at offset 0x24.

4. Accelerator RTL Design

4.1. stats_accel_core.sv

The computational engine is implemented in stats_accel_core.sv. The module is parameterized by MAX_WORDS, which is 1024 in the final design, and by ADDR_W = clog2(MAX_WORDS). Its external interface is deliberately narrow: start, clear_done, length_words, and threshold arrive as control inputs; busy and done are status outputs; the BRAM interface consists of bram_en, bram_addr, and bram_rdata; and the results are exposed as sum, minval, maxval, countgt, and cyclecount.

The core uses a three-state finite-state machine: S_IDLE, S_ISSUE, and S_CONSUME. In S_IDLE the accelerator waits for a valid start request. A run can begin only when start is asserted and length_words is in the range 1 to MAX_WORDS. On a valid start, the core latches the requested length and threshold into internal registers, clears all result registers, resets the cycle counter, clears done, sets busy, and initializes the index state for the first BRAM access.

The two-step ISSUE/CONSUME sequence is used to match the synchronous BRAM read timing. In S_ISSUE the core asserts bram_en and presents the current word index on bram_addr. In the same cycle it also copies idx into pending_idx. On the following clock, the state advances to S_CONSUME and processes bram_rdata. This structure assumes the block memory generator returns the addressed word one cycle after the read request, which matches the usual synchronous BRAM behavior.

In S_CONSUME, the datapath performs all statistic updates for the returned word. The sum path zeroextends the 32-bit sample to 64 bits before adding it into the running accumulator. Minimum and maximum are

handled carefully. On the first consumed word, identified by pending_idx == 0, both minval and maxval are set directly to the sample value. For subsequent words, bram_rdata is compared against the running extrema and the registers are updated only when a new minimum or maximum is found. The threshold counter increments when bram_rdata > threshold_latched, which implements the strict greaterthan definition used by both the README and the software application.

The core does not increment the main address index until the consume phase has completed. After processing a word, it checks whether pending_idx + 1 is still less than length_latched. If more words remain, idx is advanced to the next word index and the state returns to S_ISSUE. Otherwise the current sample was the last valid element in the dataset, so the core deasserts busy, asserts done, and returns to S_IDLE. This comparison is the mechanism that prevents reads beyond LENGTHWORDS - 1. The final word processed is exactly the word whose index is length_latched - 1.

The cycle counter is implemented as a simple free-running counter gated by busy. At every clock edge, if busy is already high, cyclecount increments by one. Because of that placement in the sequential logic, the counter records the number of cycles spent in the active run rather than the number of data elements alone. For the current FSM structure, the counted interval includes the alternating S_ISSUE and S_CONSUME cycles. The counter is cleared at the start of a run and exposed directly to software after completion.

The done and busy behavior is also explicit in the RTL. busy indicates that a run is in progress. done remains set after completion until software writes the clear_done control pulse. The clear_done path is independent of the start path, so software can explicitly clear an old completion indication before issuing a new run. On reset, the result registers are initialized to sum = 0, minval = 0xFFFFFFFF, maxval = 0, countgt = 0, and cyclecount = 0. Those reset values are superseded by the first valid run, which reinitializes the registers before any BRAM data is consumed.

4.2. stats_accel_axil.sv

The AXI-Lite wrapper in stats_accel_axil.sv turns the computational core into a memory-mapped peripheral. The interface uses a 32-bit data bus and a 6-bit address bus, which is enough to decode the implemented register set from offset 0x00 through 0x24 on word boundaries. The wrapper contains writable storage only for LENGTHWORDS and THRESHOLD. Control bits are treated as pulses rather than persistent register state.

Write handling is implemented with a small pair of hold registers. AXI-Lite allows the write address and write data channels to arrive independently, so the wrapper captures AW and W transactions into awaddr_q, wdata_q, and wstrb_q. Once both halves of a write are present and no write response is outstanding, the wrapper decodes the address and performs the requested action. This is adequate for a low-bandwidth control peripheral and keeps the interface logic compact while remaining AXI-Lite compliant at the single-transaction level.

Two details in the wrapper are particularly important. First, byte write strobes are honored for writable registers through the apply_wstrb function, which merges only the enabled byte lanes into the existing register value. Second, writes to the CONTROL address do not store a control register image. Instead, bit 0 generates a one-cycle start_pulse and bit 3 generates a one-cycle clear_done_pulse. When software later reads the CONTROL register, the returned value is synthesized from live status signals: bit 1 reflects core_done, bit 2 reflects core_busy, and bits 0 and 3 read back as zero.

The read path is handled through the reg_read function. Each register address is decoded from addr[5:2], since the bottom two bits are byte offsets within a 32-bit word. VERSION is implemented as the constant 0x00010000. The 64-bit sum is split into SUMLO and SUMHI for software access over the 32-bit bus. MINVAL, MAXVAL, COUNTGT, and CYCLECOUNT are returned directly from the core result signals.

The BRAM connection is also finalized in the wrapper. bram_clk is tied to s_axi_aclk, bram_rst is tied low, bram_we is forced to all zeros, and bram_wdata is tied to zero, confirming that the accelerator never writes dataset memory. The core generates a word index, but the external BRAM address bus is byte addressed, so the wrapper shifts the index left by two bits before driving bram_addr. The wrapper therefore bridges three domains at once: AXI-Lite register access from software, direct synchronous BRAM reads for the datapath, and result/status return to the processor.

5. Register Map

Table 1 summarizes the software-visible register map implemented by the AXI-Lite wrapper. All registers are 32 bits wide and use byte offsets from the accelerator base address 0x00008000. The practical behavior column reflects the actual RTL semantics rather than generic AXI-Lite behavior.

Offset	Register	Access	Practical function
0x00	CONTROL	R/W	bit 0 writes a start pulse; bit 1 reads done; bit 2 reads busy; bit 3 writes a clear_done pulse. Bits 0 and 3 are action bits, not sticky state.
0x04	LENGTHWORDS	R/W	Number of 32-bit words to process. A run starts only if the value is between 1 and 1024 inclusive.
0x08	THRESHOLD	R/W	Threshold used by the strict comparison value > threshold.
0x0C	VERSION	R	Read-only constant 0x00010000 used to identify the peripheral

			version.
0x10	SUMLO	R	Lower 32 bits of the 64bit sum after completion.
0x14	SUMHI	R	Upper 32 bits of the 64bit sum after completion.
0x18	MINVAL	R	Minimum value observed in the processed dataset.
0x1C	MAXVAL	R	Maximum value observed in the processed dataset.
0x20	COUNTGT	R	Count of processed values strictly greater than THRESHOLD.
0x24	CYCLECOUNT	R	Number of clock cycles accumulated while busy was high during the run.

Table 1. Accelerator AXI-Lite register map.

In practice, software uses only three kinds of writes to control a run: writing LENGTHWORDS, writing THRESHOLD, and then writing CONTROL. The provided application writes 0x8 to CONTROL to clear any stale done flag and then writes 0x1 to CONTROL to generate the start pulse. Completion is detected by polling CONTROL until bit 1 becomes one. All result registers are then read as ordinary memory-mapped words.

6. Software Application

The Vitis application in hello_world/src/helloworld.c is the software control plane for the design. It begins with init_platform() and then disables the data cache by calling Xil_DCacheDisable(). The code defines two fixed base addresses: ACCEL_BASE = 0x00008000 and BRAM_BASE = 0xC0000000. Those values match the Vivado address assignments shown in the address editor screenshot and are used directly in the subsequent Xil_Out32 and Xil_In32 transactions.

The application uses a dataset length n = 1024 and a threshold value of 500. It initializes the BRAM contents by writing the value v = i + 100 to each word location. While doing so, it also computes the software reference results in the same loop: a 64-bit sum, a minimum, a maximum, and the count of values greater than the threshold. This makes the software model deterministic and removes any dependency on external input data during verification.

After BRAM initialization, the software programs the accelerator by writing n to REG_LENGTH and the threshold to REG_THRESHOLD. It then writes 0x8 to REG_CONTROL to clear the done flag and 0x1 to REG_CONTROL to start the run. Completion is checked with a polling loop that repeatedly reads REG_CONTROL and tests bit 1. The accelerator therefore operates as a polled peripheral in the final system; no custom interrupt path is used by the application.

When the run finishes, the application reads SUMLO and SUMHI, reconstructs the full 64-bit hardware sum, and then reads MINVAL, MAXVAL, COUNTGT, and CYCLECOUNT. The result printout is arranged as a software-versus-hardware comparison table on the UART console. Finally, the code compares the reconstructed hardware results against the software reference values and prints VERIFICATION PASSED or VERIFICATION FAILED.

This software flow demonstrates the complete control loop expected of a processor-attached accelerator: shared-memory dataset preparation, register programming, start sequencing, completion detection, result collection, and cross-checking against a golden software computation. It also makes the design easier to debug because any mismatch shows up as a register-level discrepancy between the software model and the hardware output registers.

7. Address Map and Memory Organization

The address editor view in Figure 2 shows the key regions relevant to the project. The custom accelerator is mapped at 0x00008000 with a 4 KB range. The software only uses offsets 0x00 through 0x24 within that region, but Vivado assigns a larger AXI-Lite aperture than the minimum register footprint. This is normal for a custom peripheral and does not change the functional register map seen by software.

The BRAM used as the dataset store is mapped at 0xC0000000 with an 8 KB range through the AXI BRAM Controller. The current application processes 1024 words, which corresponds to 4096 bytes of active data. The mapped BRAM region is therefore larger than the specific dataset used in the benchmark. That larger mapping is not an inconsistency. It simply means the memory block can hold more data than the current test case consumes, while the accelerator itself still enforces a maximum valid LENGTHWORDS of 1024 words.

The address map also includes DDR3 through the MIG interface, UARTLite, and other standard peripherals. For this project, however, the accelerator dataset path stays on BRAM rather than external DDR. That choice keeps the datapath local to the processor and accelerator and matches the final design contract described in the README.

Base address	Range	Block	Role in this project

0x00008000	4 KB	stats_accel_axil_0	AXI-Lite control/status register block for the custom accelerator.
0xC0000000	8 KB	axi_bram_ctrl_0 / BRAM	Shared dataset memory. MicroBlaze writes through AXI BRAM Controller; accelerator reads through BRAM port B.
0x40600000	64 KB	axi_uartlite_0	UART console used by the standalone application for benchmark output.

Table 2. Key address regions used by the standalone application.

Figure 2 shows the corresponding Vivado address assignments. The screenshot confirms the two addresses used directly in software: 0xC0000000 for the BRAM region and 0x00008000 for the accelerator register block.

Diagram | Address Editor | Address Map | IP Catalog | IP Catalog (2) | bd_wrapper.v

Assigned (20) | Unassigned (0) | Excluded (0) | Hide All

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/microblaze_0/Data, /microblaze_0/Instruction)					
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	32K	0x7FFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	128M	0x87FF_FFFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	128M	0x87FF_FFFF
/stats_accel_axil_0/s_axi	s_axi	reg0	0x8000	4K	0x8FFF
/stats_accel_axil_0/s_axi	s_axi	reg0	0x8000	4K	0x8FFF
/microblaze_0/Instruction (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	32K	0x7FFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	128M	0x87FF_FFFF
/stats_accel_axil_0/s_axi	s_axi	reg0	0x8000	4K	0x8FFF

Figure 2. Vivado address editor view for the MicroBlaze system. The accelerator register block is mapped at 0x00008000 and the shared BRAM is mapped at 0xC0000000.

8. Verification and Demonstration

Correctness in the final system is checked by direct software-versus-hardware comparison. The Vitis application writes a known dataset into BRAM, computes the reference statistics in software while initializing that dataset, and then compares the hardware result registers against the software values after the accelerator finishes. A run is considered successful only when the 64-bit sum, minimum, maximum, and greater-than-threshold count all match the reference model. The application also reports the hardware cycle counter so that functional verification and timing observation are available in the same console session.

This verification method exercises the actual deployed SoC path rather than an isolated unit test. It includes processor writes to BRAM, register programming over AXI-Lite, BRAM reads by the accelerator, and result reads back into software. A demonstration of the final board-level system is available at the following link: https://www.youtube.com/watch?v=YFce6t9Ztq0. The video is best viewed as supporting evidence that the integrated design runs on hardware; the main technical content remains the SoC architecture and control flow described in this report.

9. Conclusion

The final project demonstrates a complete processor-attached hardware accelerator implemented in a MicroBlaze SoC on the AMD Urbana board. The design combines a custom RTL statistics engine, an AXI-Lite register interface, a dual-port BRAM datapath, and a standalone Vitis application that drives and verifies the hardware directly on the board.

From a hardware perspective, the project shows how a custom peripheral can be integrated into the AXI fabric while avoiding unnecessary control-path overhead for bulk data. The BRAM is written by MicroBlaze through the AXI BRAM Controller but read by the accelerator through its own memory port, which is a clean and efficient organization for this type of workload. From a software perspective, the project shows how a soft processor can initialize memory, program a custom IP block, poll for completion, reconstruct multiword results, and validate the hardware against a reference model. Taken together, the design is a

clear example of software-controlled hardware acceleration in a MicroBlaze-based SoC.